

Resumo de Programação Orientada a Objetos (POO)

Paradigmas de Programação

Imperativo: foco em comandos sequenciais (C).

Funcional: foco em funções puras e imutabilidade (Haskell).

Orientado a Objetos (POO): foco em objetos que representam entidades do mundo real (Java, Python, C#).

Vantagens de POO: reutilização de código (herança), organização, manutenção e extensão facilitadas, encapsulamento e colaboração em equipe.

Desvantagens: curva de aprendizado maior, complexidade para problemas simples, exige planejamento inicial.

Classes e Objetos

Classe: modelo que define atributos e métodos de um tipo de objeto.

Objeto: instância concreta de uma classe, com valores próprios.

Exemplo:

```
public class Carro {  
    String cor;        // atributo: característica do carro  
    int ano;           // atributo: ano do carro  
  
    void buzinar() { // método: ação que o carro pode executar  
        System.out.println("Biiiii!");  
    }  
}  
  
// Criando objeto  
Carro meuCarro = new Carro(); // instânciação: cria o objeto  
meuCarro.cor = "Azul";        // atribuindo valor ao atributo  
meuCarro.ano = 2022;  
meuCarro.buzinar();           // chamando método do objeto
```

Memória em Java

- **Stack (pilha):** guarda variáveis locais e chamadas de método; rápida e limitada.
- **Heap (monte):** guarda objetos criados com `new`; gerada pelo Garbage Collector; mais lenta, mas com mais espaço.

```
Carro carro = new Carro(); // 'carro' é a referência na stack
                        // objeto Carro() está na heap
```

Diagrama UML

Diagrama de Classe: modelo abstrato da estrutura (atributos e métodos).

Diagrama de Objeto: instância com valores concretos.

Exemplo de diagrama de objeto:

```
meuCarro : Carro
cor = "Vermelho"
ano = 2020
```

Escopo de Variáveis

Local: dentro de métodos; existe só enquanto o método executa.

Instância (atributo): dentro da classe; cada objeto tem sua cópia.

Bloco: dentro de blocos `{}`; existe só dentro do bloco.

```
public class Exemplo {
    int x; // variável de instância

    void metodo() {
        int y = 10; // variável local
        if(true) {
            int z = 20; // variável de bloco
        }
    }
}
```

Encapsulamento

Princípio: ocultar detalhes internos e controlar acesso.

Modificadores em Java:

`private` → só dentro da própria classe

`public` → acesso liberado

`protected` → pacote ou classes herdeiras

`default` → pacote

```
public class ContaBancaria {
    private double saldo;           // atributo privado

    public double getSaldo() {      // getter: acesso controlado
        return saldo;
    }

    public void depositar(double valor) { // método controlado
        if(valor > 0) saldo += valor;
    }
}
```

Membros de Classe

Instância: cada objeto tem sua cópia.

Estático (static**):** pertence à classe, compartilhado entre objetos.

```
public class Calculadora {
    public static double PI = 3.14159;      // atributo estático
    public static int somar(int a, int b) {  // método estático
        return a + b;
    }
}

// Uso sem criar objeto
double valor = Calculadora.PI;
int resultado = Calculadora.somar(5, 3);
```

Sobrecarga de Métodos

Permite múltiplos métodos com **mesmo nome** e **assinaturas diferentes** (quantidade ou tipo de parâmetros).

```
public class Impressora {
    public void imprimir(String texto) { System.out.println(texto); } // método 1
    public void imprimir(int numero) { System.out.println(numero); } // método 2
    public void imprimir(double numero) { System.out.println(numero); } // método 3
}

// Uso
Impressora imp = new Impressora();
imp.imprimir("Olá"); // escolhe método 1
imp.imprimir(123);   // escolhe método 2
imp.imprimir(45.6);  // escolhe método 3
```

Construtores

Método especial para **inicializar objetos**.

Mesmo nome da classe, **sem tipo de retorno**.

Executado automaticamente com `new`.

```
public class Pessoa {
    private String nome; // atributo
    private int idade;   // atributo

    public Pessoa(String nome, int idade) { // construtor
        this.nome = nome;
        this.idade = idade;
    }

    public String getNome() { return nome; } // getter
    public int getIdade() { return idade; } // getter
}

// Uso
Pessoa p1 = new Pessoa("Ana", 25);
System.out.println(p1.getNome()); // imprime "Ana"
```

Sobrecarga de Construtores

```
public class Retangulo {
    private int largura, altura;

    public Retangulo(int largura, int altura) { // construtor 1
        this.largura = largura;
        this.altura = altura;
    }

    public Retangulo(int lado) { // construtor 2 (quadrado)
        this.largura = lado;
        this.altura = lado;
    }
}

// Uso
Retangulo r1 = new Retangulo(10, 5);
Retangulo r2 = new Retangulo(7); // quadrado
```